

KARA Eren



PROJET ESCAPE GAME

01

Partie
concernée

02

Planification

03

Modélisation

04

Préparation

05

Commande

06

Prototypage

07

Test

08

Validation

Problématique

Une société spécialisée dans l'organisation d'événements souhaite enrichir son offre avec des jeux interactifs autour de thèmes actuels et populaires. Dans le cadre de son nouveau thème "Escape Futuriste", elle cherche à proposer un objet électronique original de la taille d'une petite table.

- **Objectif du projet :**

Concevoir une interface interactive et visuelle, basée sur un bandeau de LED RGB et des capteurs tactiles capacitifs, capable de réagir en temps réel au toucher de l'utilisateur, tout en étant contrôlable à distance via Wi-Fi à l'aide d'un microcontrôleur ESP32 .

Diagramme de séquences

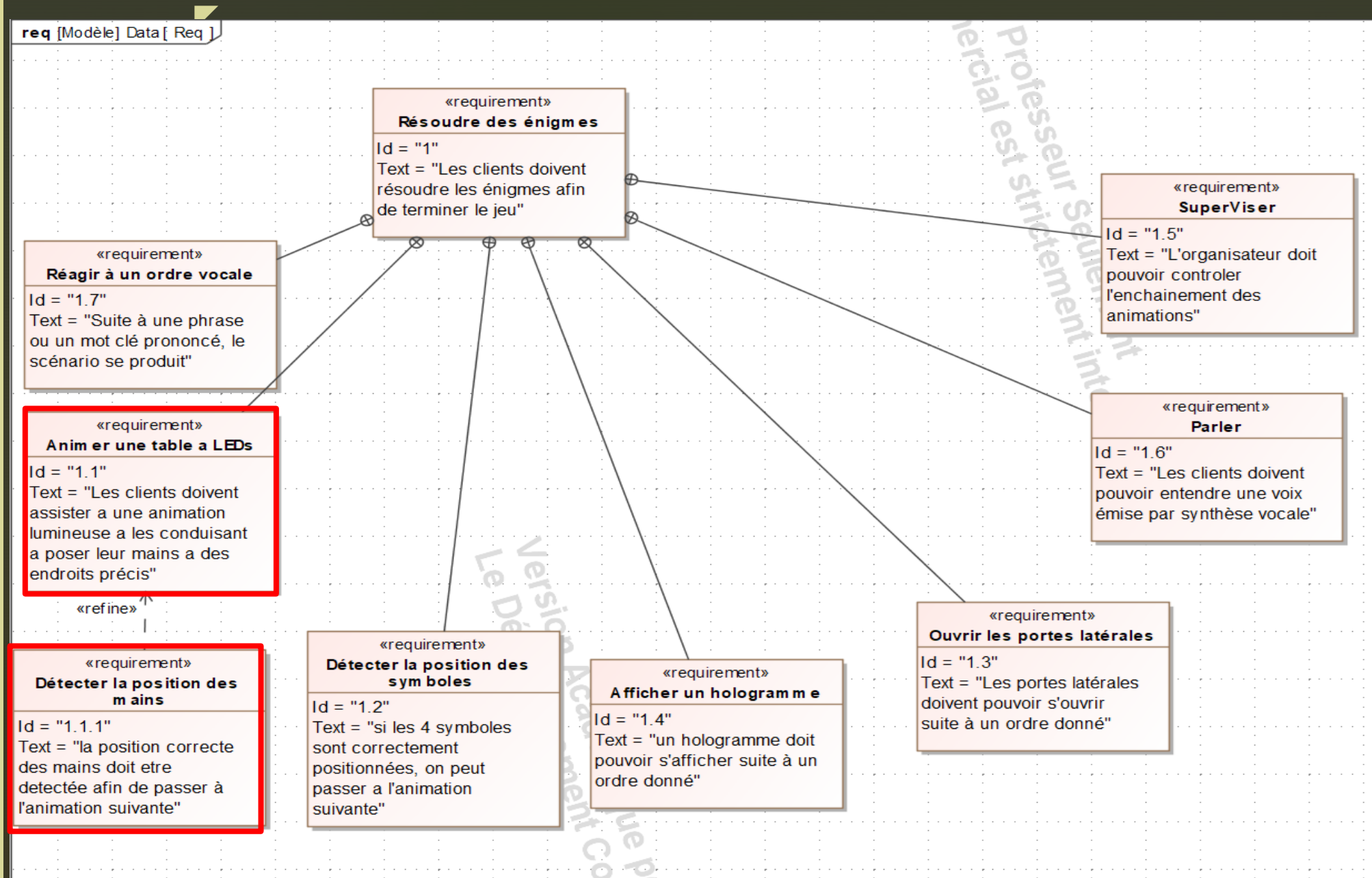
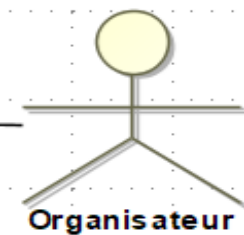
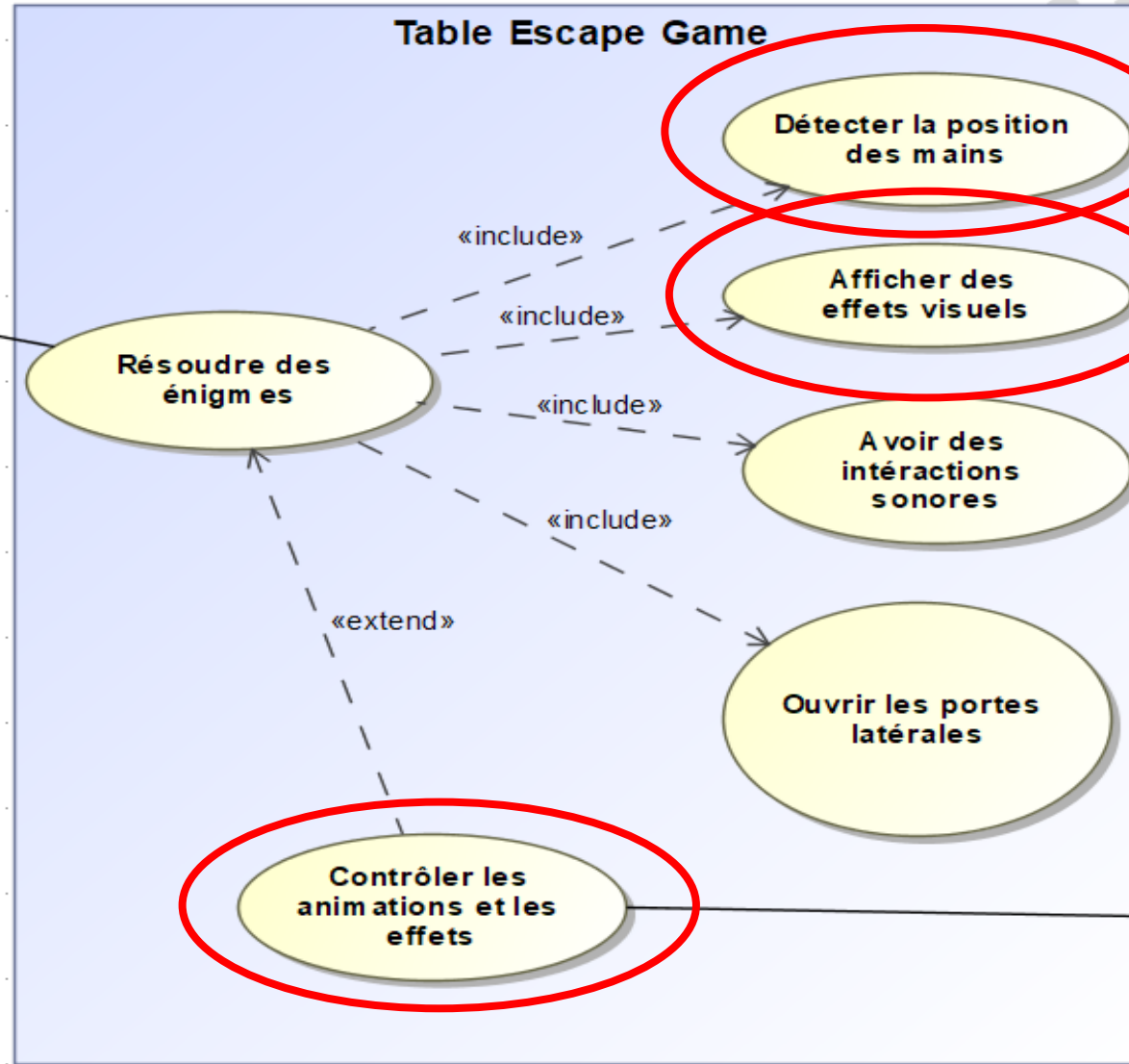
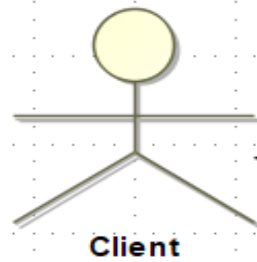
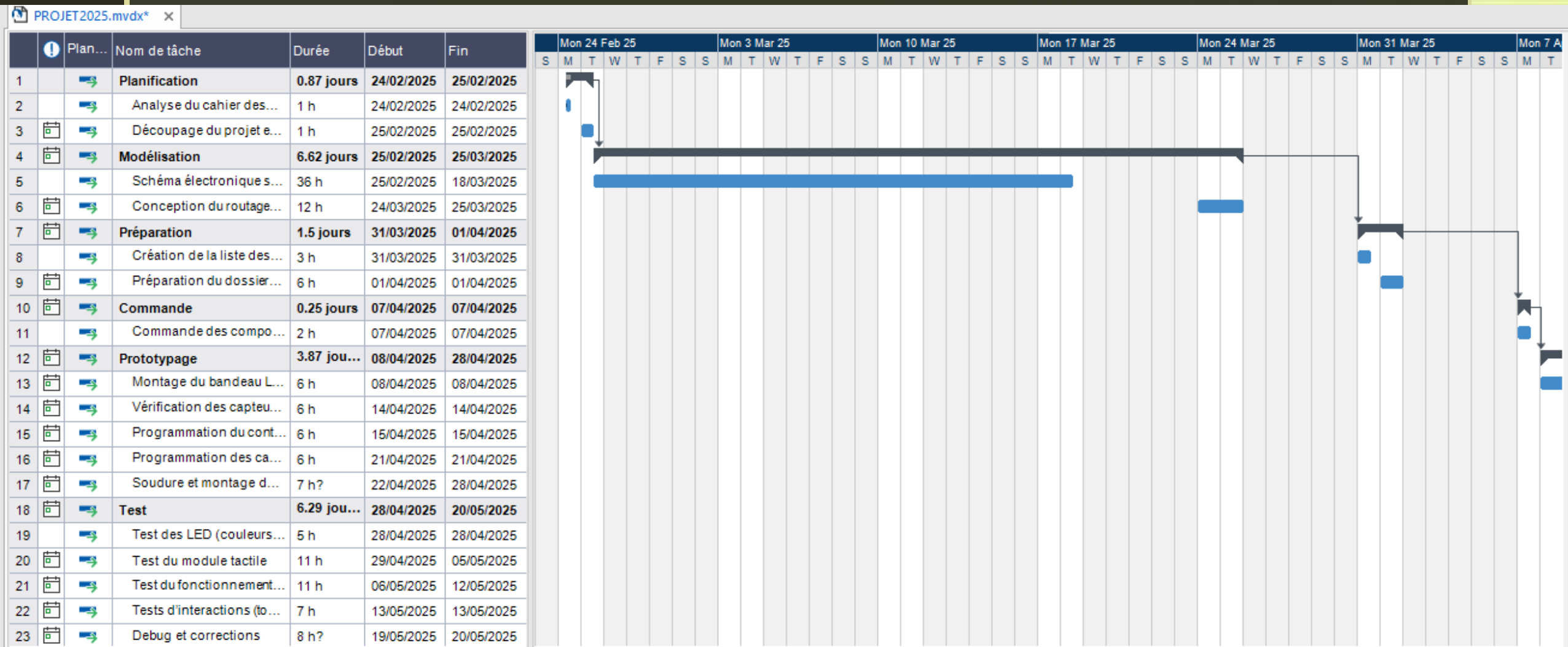


Diagramme de cas d'utilisation

uc [Model] Data [Use Case]



Planification 1/2



Planification 2/2

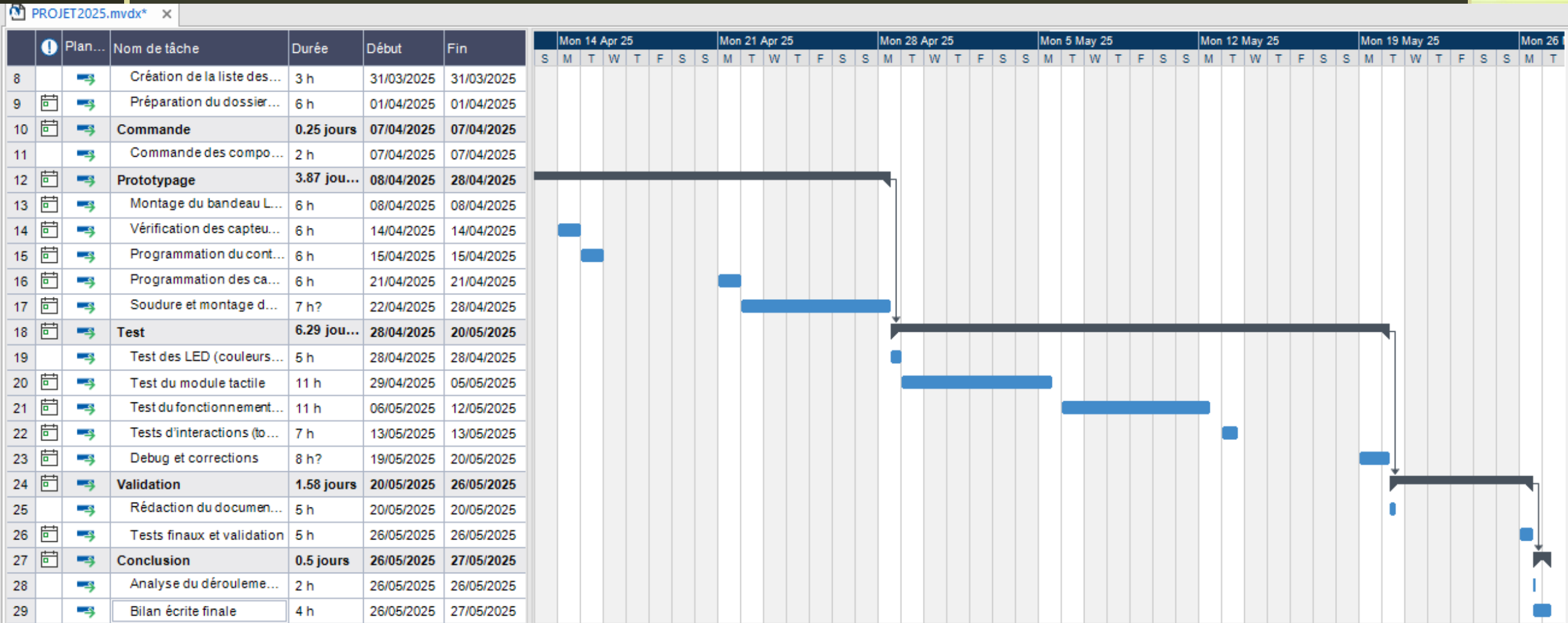


Diagramme de Séquence

1/2



Diagramme de Séquence

2/2

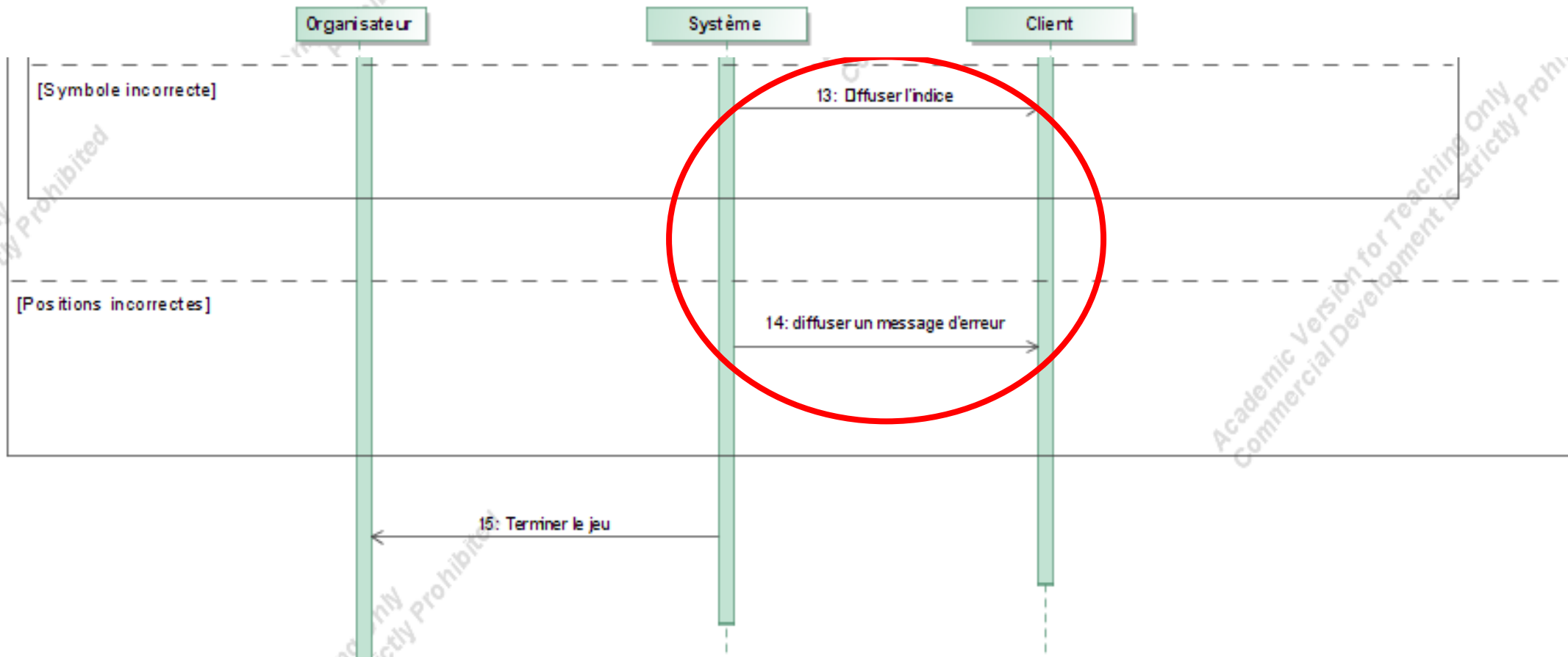
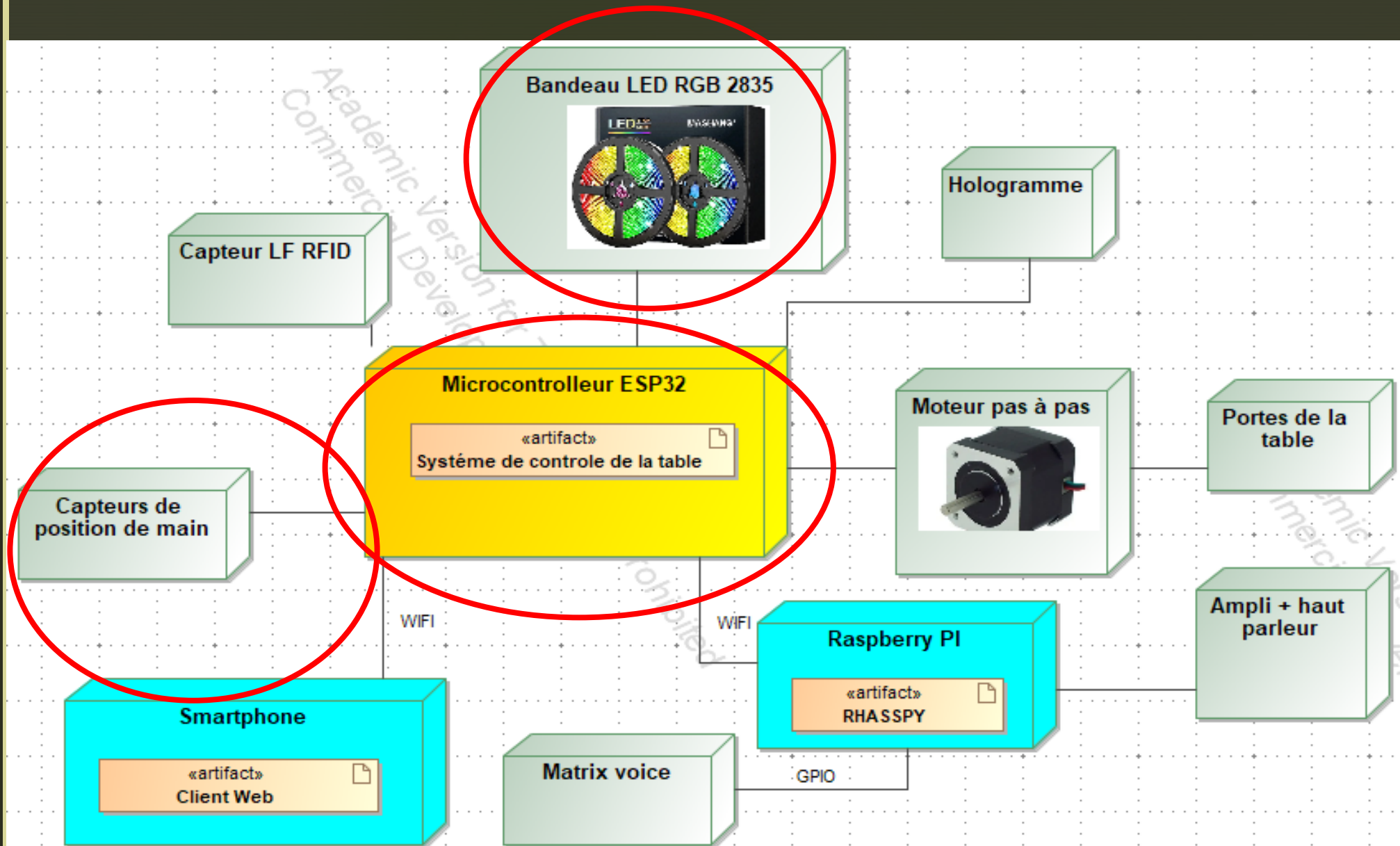
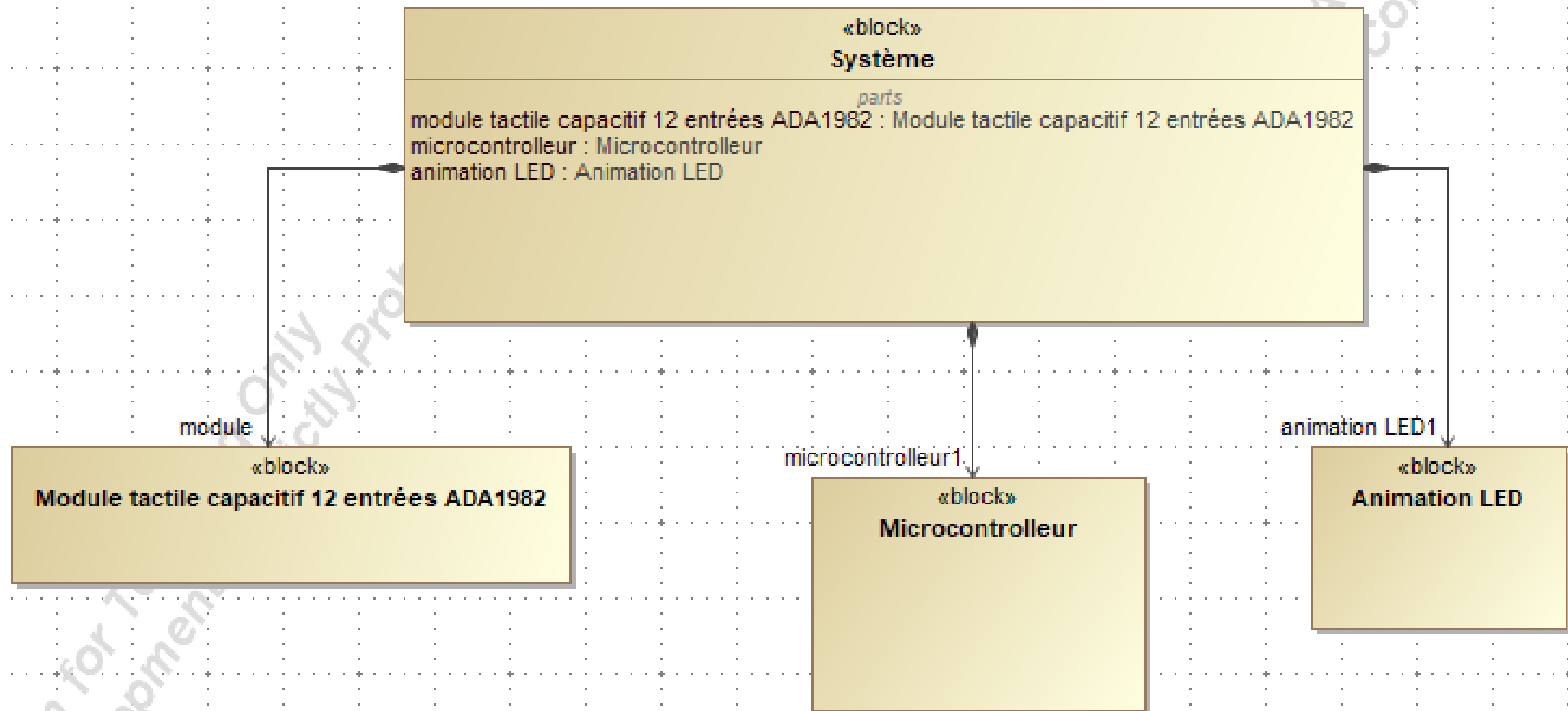


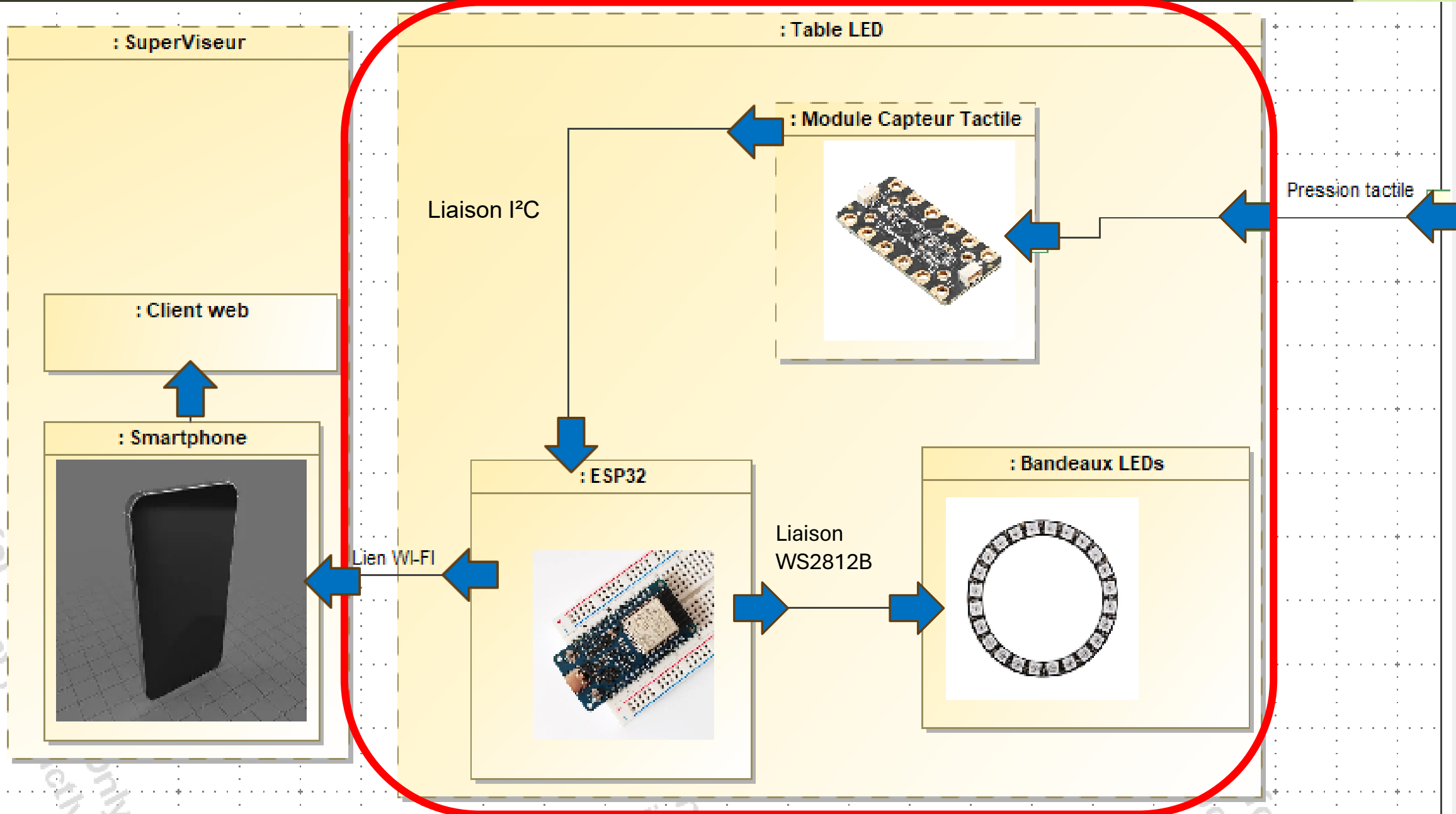
Diagramme de Déploiement



Modélisation : Block Definition Diagram

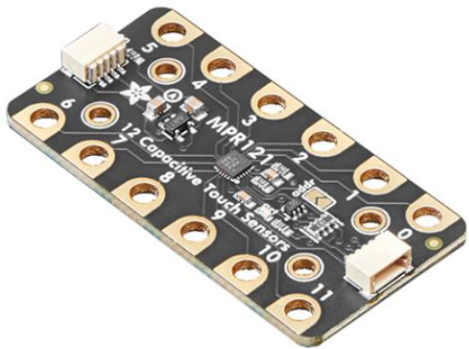


Modélisation : Internal Block Diagram

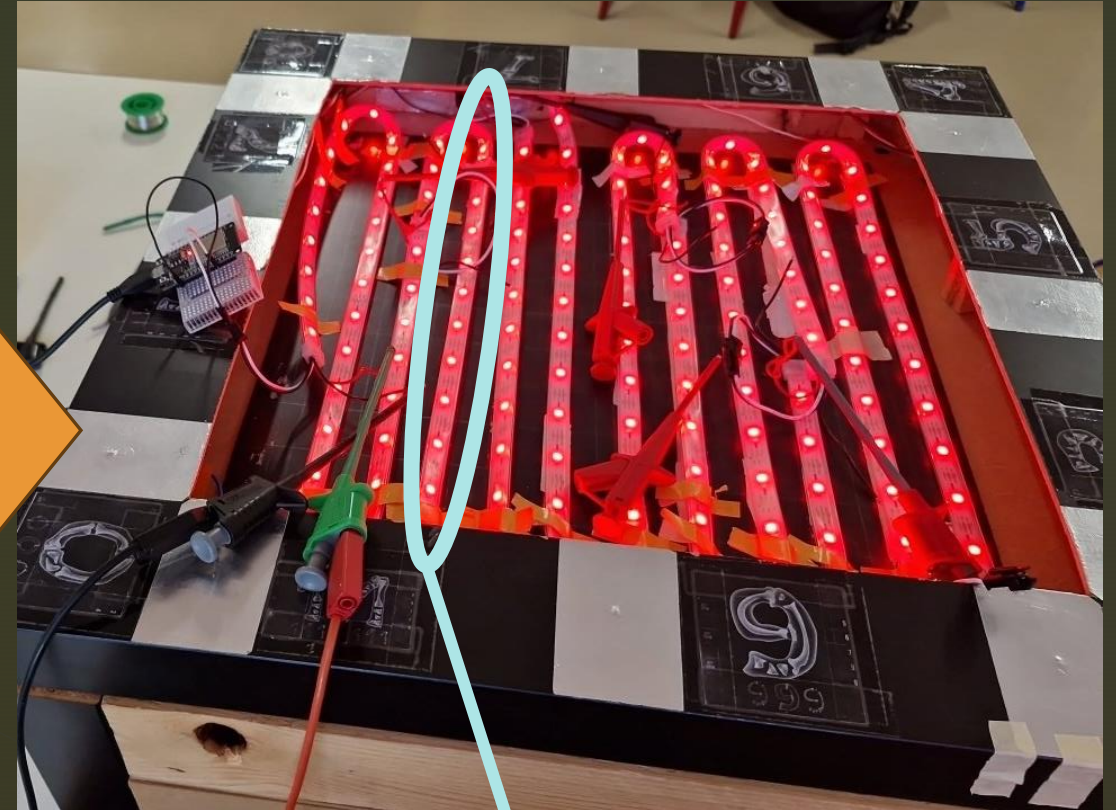


Commande

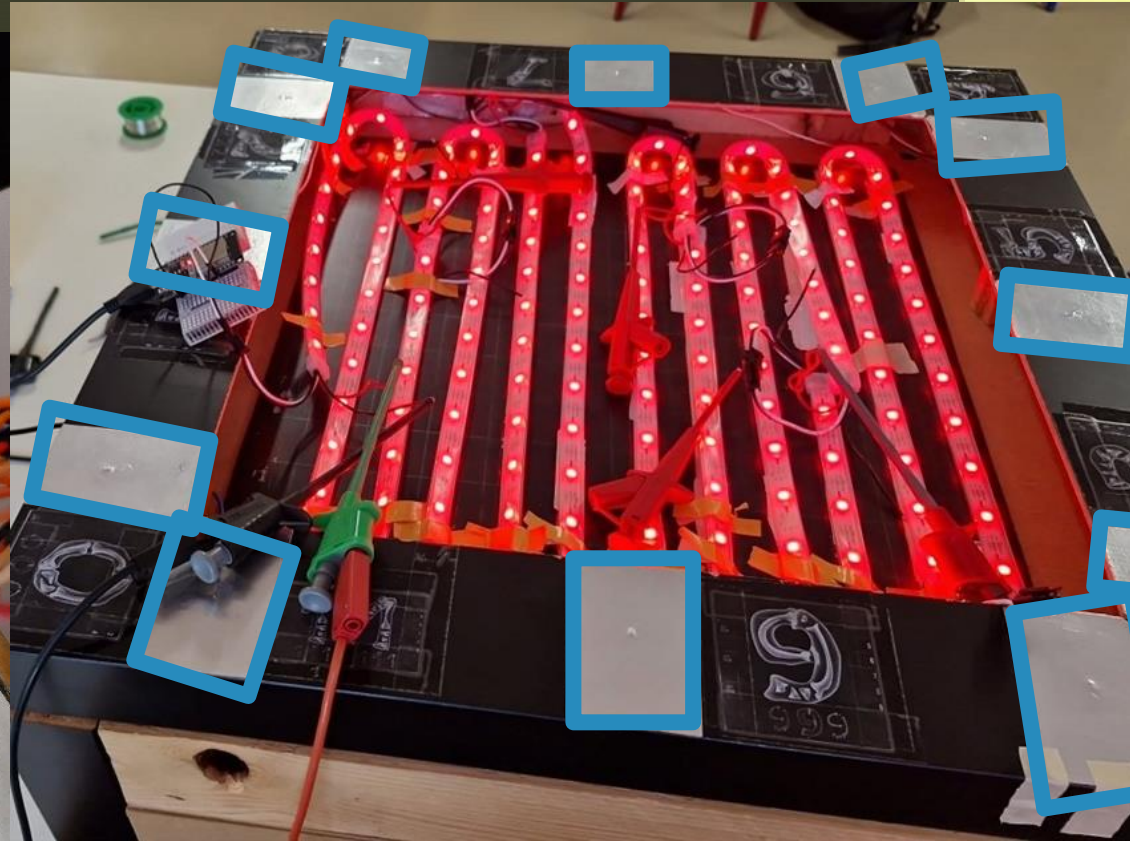
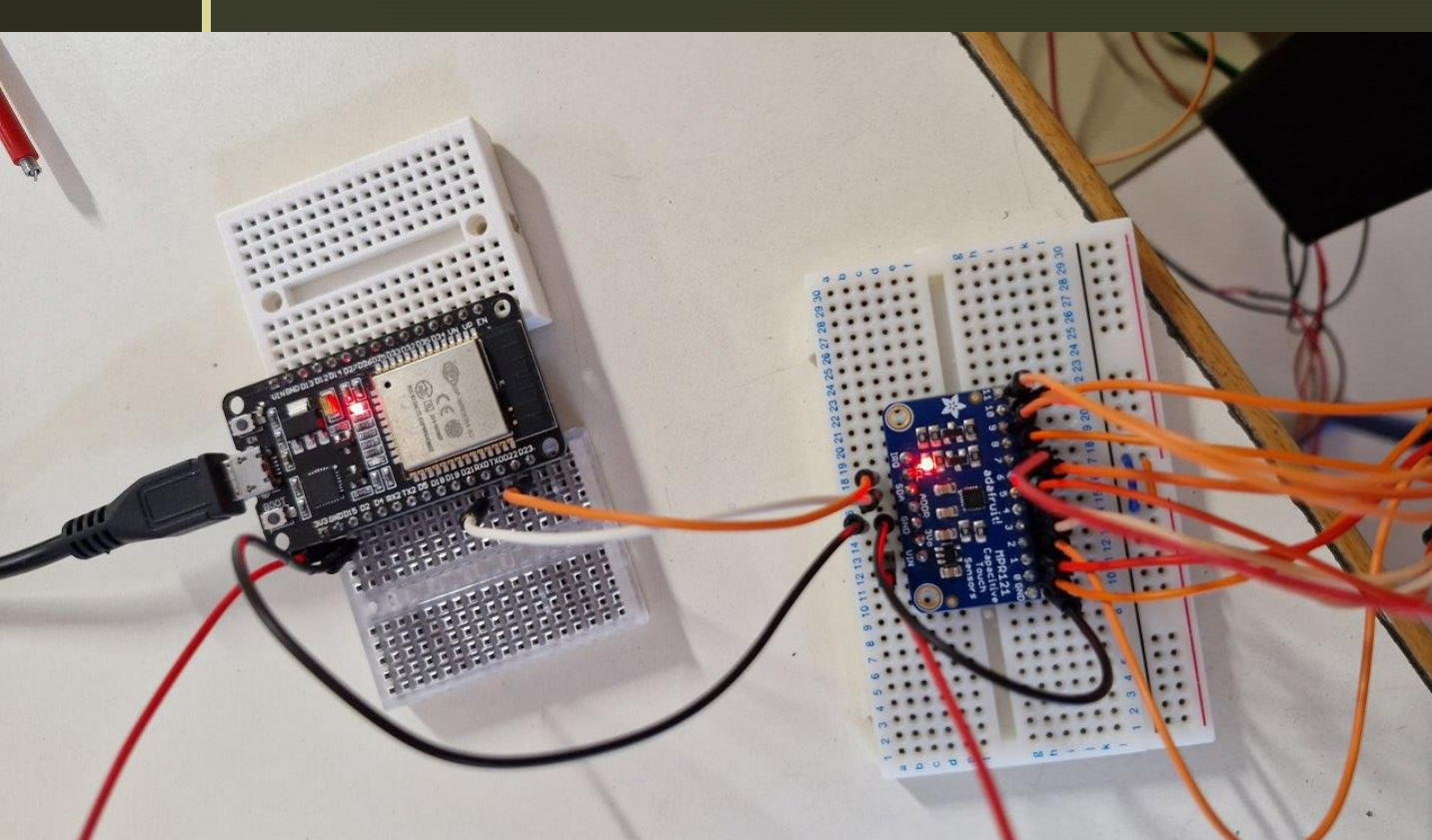
Description	Quantité	Prix unitaire	prix total	code	Lien
Module tactile capacitif 12 entrées ADA1982	2	10.20 €	20.40 €	33242	https://www.gotronic.fr/
Ruban NeoPixel RGB 1m 30 LEDs ADA1376	5	22,80 €	114 €	3880	https://www.gotronic.fr/
Boîtier NSK254-3B	5	0,06 €	0.30 €	48721	https://www.gotronic.fr/



Prototypage



Prototypage



Test

```
#include <Wire.h>           // I²C & ESP32
#include "Adafruit_MPR121.h"

Adafruit_MPR121 cap = Adafruit_MPR121();

void setup() {
  Serial.begin(9600);
  while (!Serial); // Attendre que le port série soit prêt

  // Initialiser le capteur avec les broches SDA = GPIO21 et SCL = GPIO22
  if (!cap.begin(0x5A, &Wire)) {
    Serial.println("Impossible de trouver le MPR121. Vérifiez les connexions !");
    while (1); // Boucle infinie si le capteur n'est pas détecté
  }
  Serial.println("MPR121 détecté !");
}

void loop() {
  uint16_t touched = cap.touched();

  for (uint8_t i = 0; i < 12; i++) {
    // Utilise un masque binaire pour vérifier si l'électrode 'i' est touchée
    if (touched & (1 << i)) {
      Serial.print("Électrode ");
      Serial.print(i);
      Serial.println(" touchée");
    }
  }

  delay(100); // Attendre 100 ms avant la prochaine lecture
}
```

The IDE interface shows the following components:

- SOURCE CONTROL** panel on the left with a commit button and a list of changes.
- Changes** list showing `platformio.ini` and `main...` with modification status.
- Serial Monitor** at the bottom showing a list of test results for various pins (V9, V8, V7, V6, V5, test v4, test v3, test v2, V1 allumage led).

Test

src > main.cpp > loop()

```
1 // Inclusion de la bibliothèque FastLED, qui permet de contrôler des LEDs adressables type WS2812B
2 #include <FastLED.h>
3
4 // Définition des constantes
5 #define LED_PIN      21          // Broche de données de l'ESP32 reliée au bandeau LED
6 #define NUM_LEDS     150        // Nombre total de LEDs sur le bandeau
7 #define BRIGHTNESS   100        // Luminosité des LEDs (valeur entre 0 et 255)
8 #define LED_TYPE     WS2812B    // Type de LED utilisée (ici des WS2812B)
9 #define COLOR_ORDER   GRB        // Ordre des couleurs (vert, rouge, bleu). Dépend du bandeau utilisé
```

```
10
11 // Création d'un tableau d'objets de type CRGB pour stocker l'état de chaque LED
12 CRGB leds[NUM_LEDS];
```

```
13
14 void setup() {
15     // Initialisation du bandeau LED avec les paramètres définis (type, broche, ordre)
16     FastLED.addLeds<LED_TYPE, LED_PIN, COLOR_ORDER>(leds, NUM_LEDS);
17
18     // Réglage de la luminosité générale des LEDs
19     FastLED.setBrightness(BRIGHTNESS);
20 }
21
```

```
22 void loop() {
23     fill_solid(leds, NUM_LEDS, CRGB::Red); // Remplit toutes les LEDs avec la couleur rouge
24     FastLED.show();                         // Envoie les données au bandeau LED
25     delay(1000);                           // Pause de 1 seconde
26
```

```
void loop() {
    fill_solid(leds, NUM_LEDS, CRGB::Red); // Remplit toutes les LEDs avec la couleur rouge
    FastLED.show();                       // Envoie les données au bandeau LED
    delay(1000);                          // Pause de 1 seconde

    fill_solid(leds, NUM_LEDS, CRGB::Green);
    FastLED.show();
    delay(1000);

    fill_solid(leds, NUM_LEDS, CRGB::Blue);
    FastLED.show();
    delay(1000);

    fill_solid(leds, NUM_LEDS, CRGB::Yellow);
    FastLED.show();
    delay(1000);

    fill_solid(leds, NUM_LEDS, CRGB::Purple);
    FastLED.show();
    delay(1000);

    // La boucle recommence indéfiniment, changeant la couleur toutes les 1 seconde
}
```

Test Final

Message (Ctrl+Ent...)

Publish Branch

G... Auto

final Eren master

engine succes Eren

animation debut ajo...

capteur fait afficher ...

Capteurs et leds ma...

web marche Eren

wled mpr121 Eren

test web Eren

src > main.cpp > loop()

97

98 // Variables pour mon animation de succes (le check vert)

99 int g_successCheckState = 0; // Pour gerer les etapes du check (flash, etendre, pulsation,

100 int g_successPulseCount = 0; // Pour compter combien de fois ca pulse

101 const int MAX_PULSES = 2; // Je veux que ca pulse 2 fois pour le check

102

103 // Variables pour ma sequence de capteurs (11 puis 5)

104 unsigned long lastSensor11TouchTime = 0;

105 bool sensor11TouchedRecently = false;

106 const unsigned long sequence_timeout = 1000; // J'ai 1 seconde pour faire la sequence

107

108 > // Cette fonction met a jour l'animation de mes LEDs...

110 > void updateLEDAnimation() {...

281

282 > // Cette fonction recoit mes commandes JSON pour les LEDs...

284 > void processLEDCommand(String jsonString) {...

371

372 // Ma page HTML que je sers sur le serveur web

373 const char index_html[] PROGMEM = R"rawliteral(

374 <!DOCTYPE HTML><html><head>

375 <title>Controle Table LED</title>

376 <meta name="viewport" content="width=device-width, initial-scale=1">

377 <style>

378 body { font-family: Arial; text-align: center; margin:auto; padding-top:20px;}

379 .button { background-color: #4CAF50; border: none; color: white; padding: 10px 20px;

380 text-align: center; text-decoration: none; display: inline-block;

PROBLEMS

OUTPUT

DEBUG CONSOLE

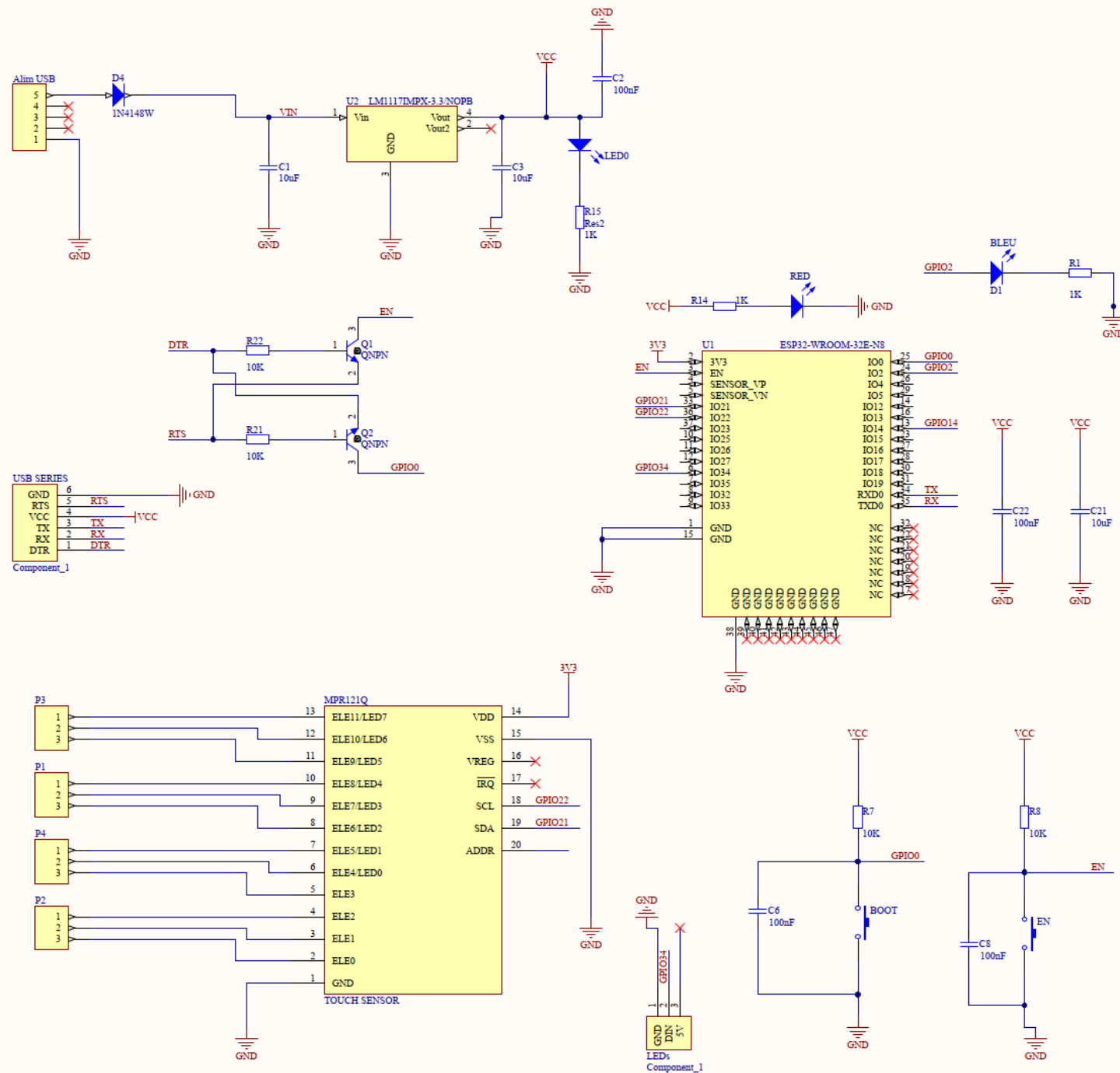
TERMINAL

PORTS

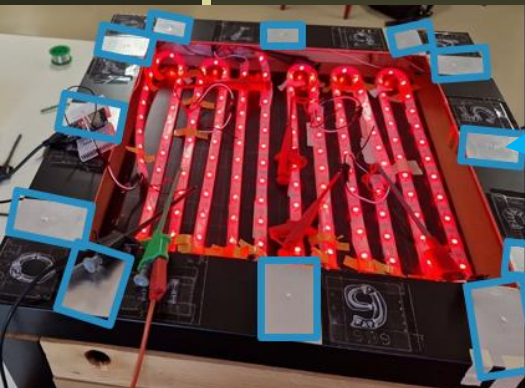
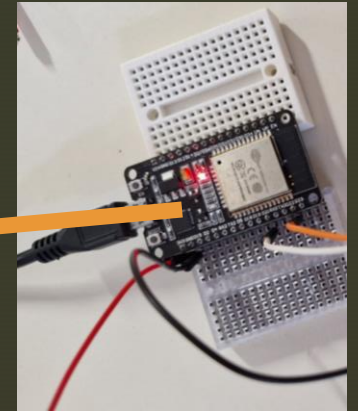
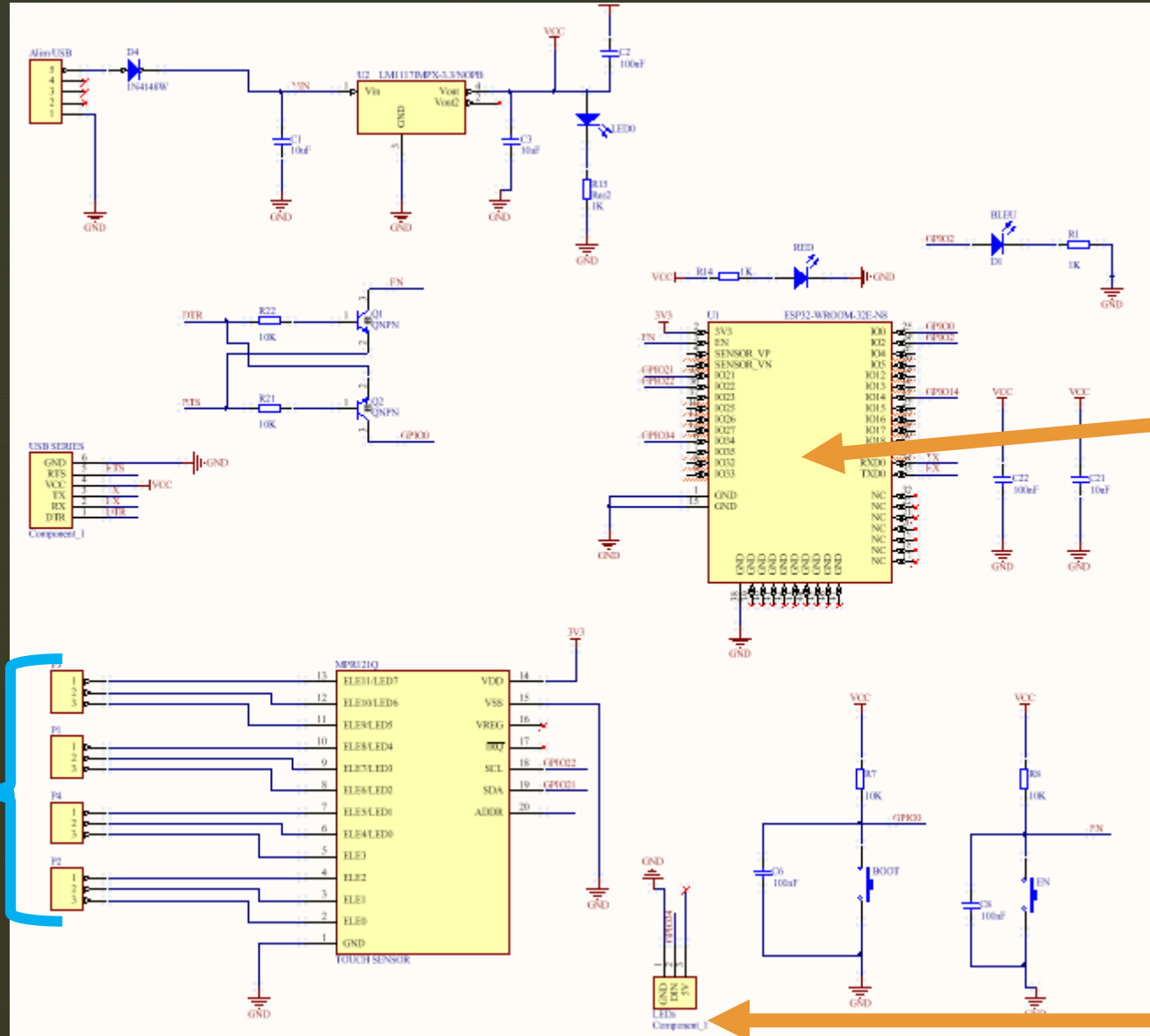
Success Animation: Flash initial vert.
Electrode 5 relachee.
Success Animation: LEDs eteintes pour pulsation.
Success Animation: Pulsation 1/2
Success Animation: Pulsation 2/2
Animation de succes terminee.

Validation

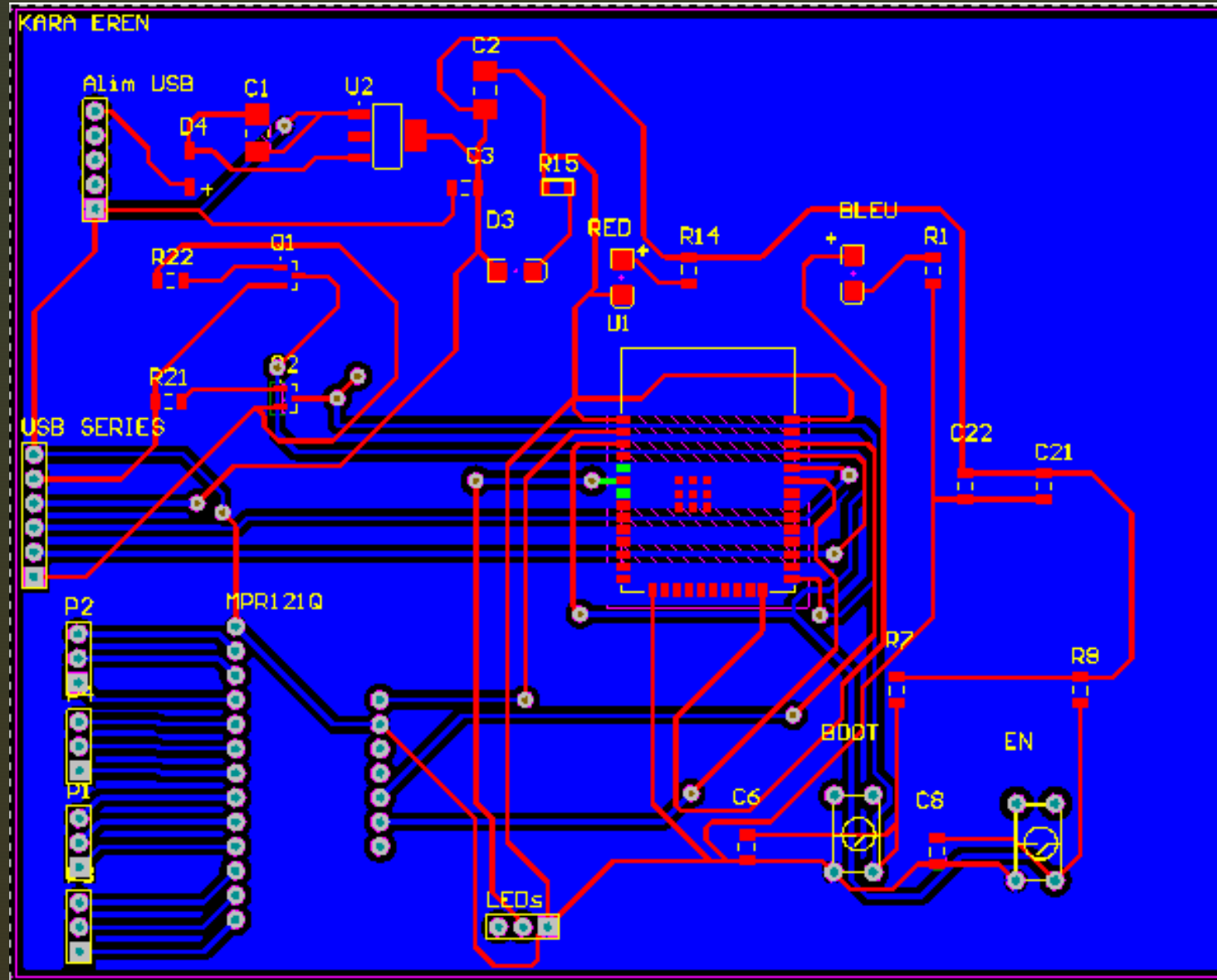
Schéma structurel 1



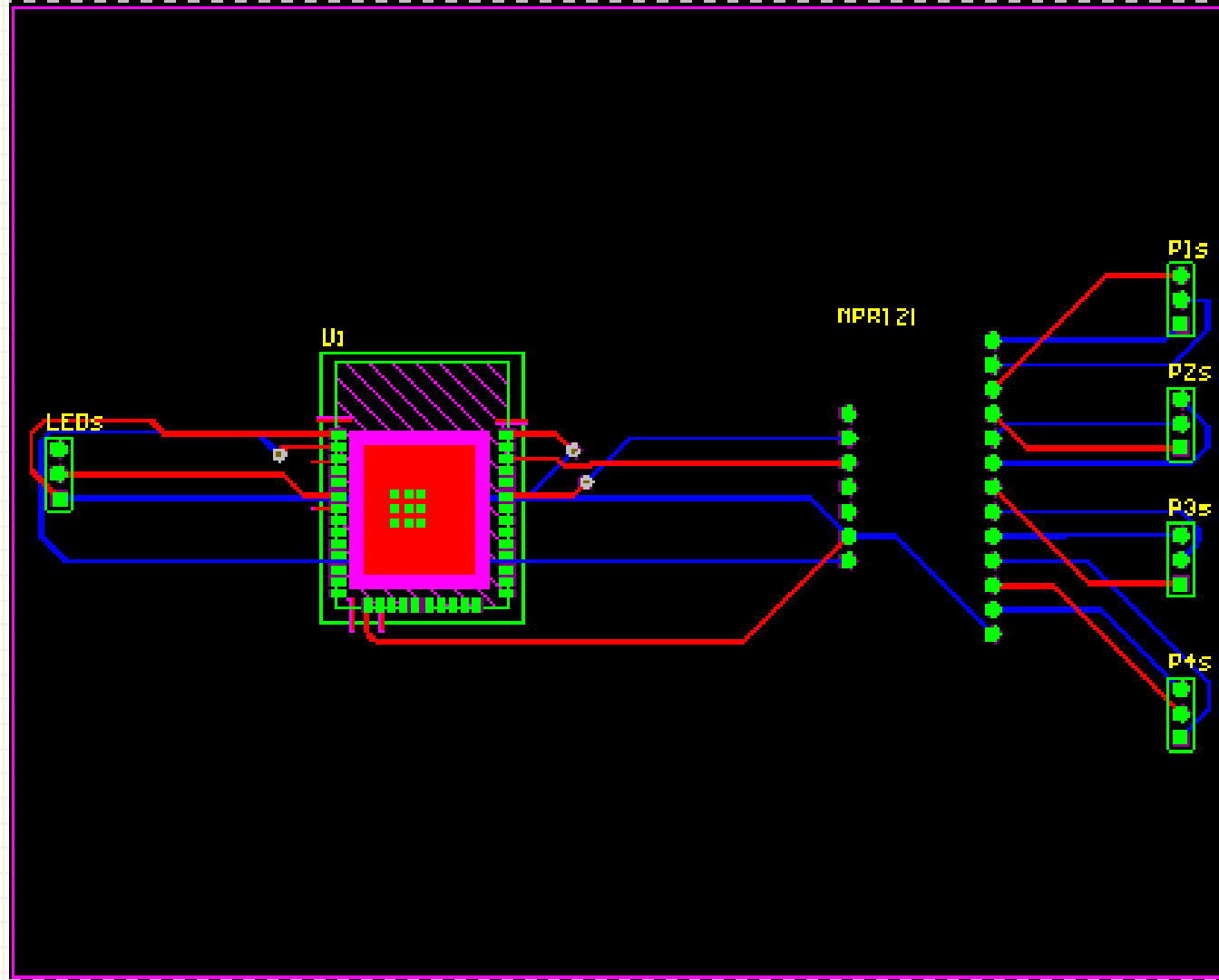
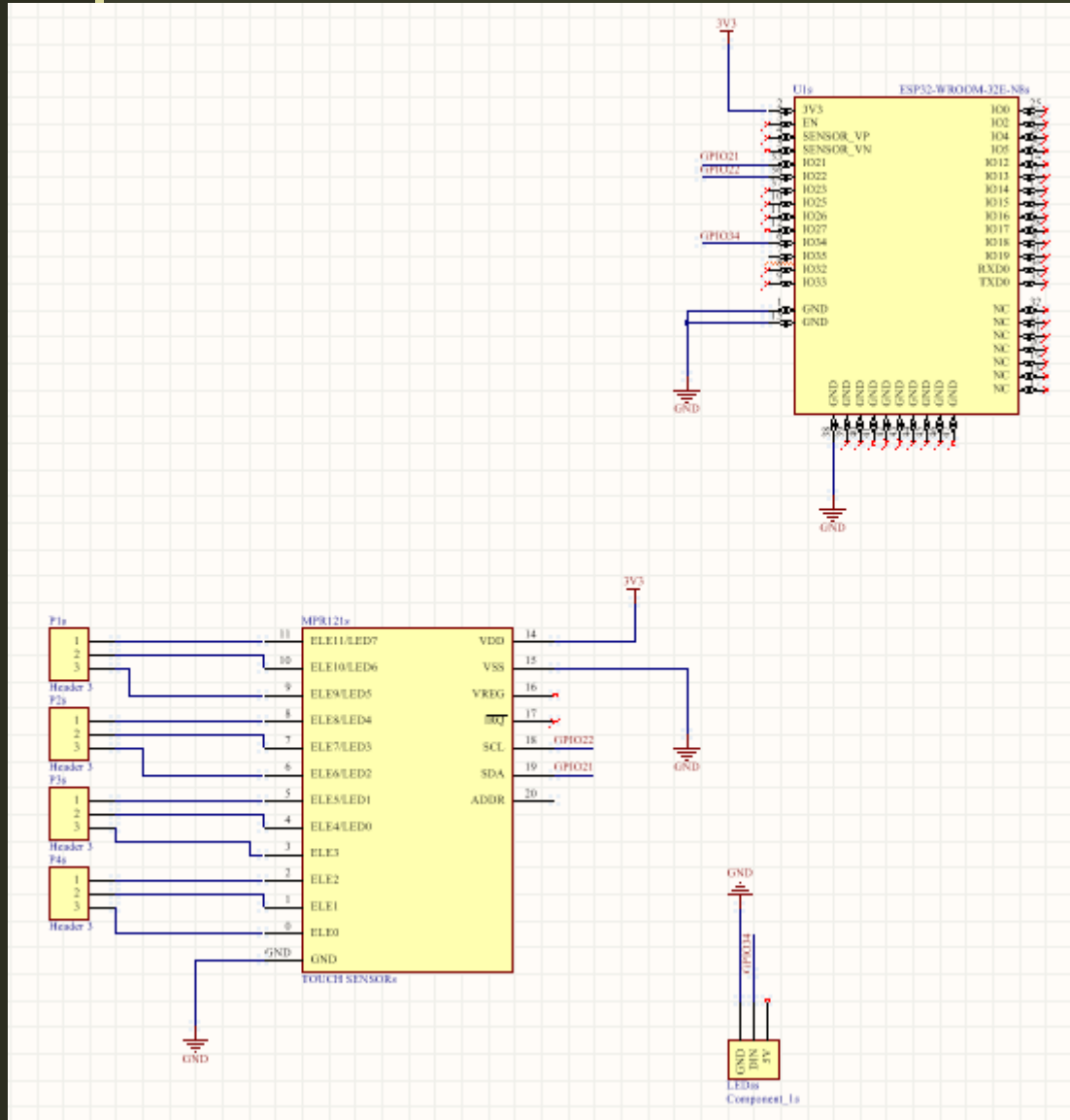
Validation



Validation ➤ PCB



Validation schéma structurel 2



Conclusion

- Ce projet nous a permis d'acquérir des compétences techniques en électronique, en développement embarqué, en gestion de projet et en communication.
- Il nous a offert l'opportunité d'innover et d'améliorer l'expérience des Escape games.
- Nous avons créé un système robuste, réactif et évolutif pouvant s'adapter à de nombreux cas d'utilisation.
- Je vous remercie de votre attention et reste à votre disposition pour vos questions.